INFORME DE RESULTADOS

Benchmark Reproducible de Multiplicación de Matrices NxN

C++ | Go | Java | Python

Experimento Computacional Reproducible
Alineado con ACM Artifact Review & Badging

Octubre 2026

1. Introducción

Este informe presenta los resultados del benchmark reproducible de multiplicación de matrices NxN, un experimento computacional diseñado para comparar el rendimiento de cuatro lenguajes de programación: C++, Go, Java y Python.

El proyecto sigue los principios de reproducibilidad y replicabilidad científica alineados con las directrices de ACM Artifact Review & Badging, garantizando que cualquier investigador pueda replicar los resultados obtenidos.

2. Objetivo

Determinar si existen diferencias estadísticamente significativas en el tiempo de ejecución del algoritmo naive de multiplicación de matrices NxN entre implementaciones equivalentes en C++, Go, Java y Python, y cómo varían estas diferencias según el tamaño de la matriz.

3. Hipótesis

H_0 (Hipótesis nula): El lenguaje de implementación no produce diferencias estadísticamente significativas en el tiempo de ejecución.

H_1 (Hipótesis alternativa): Al menos uno de los lenguajes presenta un tiempo de ejecución significativamente diferente de los demás.

4. Metodología

4.1 Algoritmo

Se implementó el algoritmo naive de multiplicación de matrices con complejidad $O(n^3)$, utilizando un triple bucle anidado. Este algoritmo fue elegido por su simplicidad y por ser idéntico en los cuatro lenguajes, eliminando diferencias de implementación algorítmica.

4.2 Generación de datos

Las matrices de entrada se generan con un Generador Lineal Congruencial (LCG) con semilla fija (seed=42 para matriz A, seed=43 para matriz B). El mismo generador con los mismos parámetros se implementó en los cuatro lenguajes para garantizar matrices de entrada idénticas.

4.3 Variables del experimento

Variable	Tipo	Valores
Lenguaje	Independiente (factor)	C++, Go, Java, Python
Tamaño de matriz (N)	Independiente (factor)	500, 1000, 1500
Tiempo de ejecución	Dependiente	Segundos (float)
Repeticiones	Control	10 por combinación

Variable	Tipo	Valores
Semilla del generador	Control	42 (fija)

4.4 Diseño experimental

Se realizaron 10 ejecuciones por cada combinación de lenguaje y tamaño de matriz, resultando en un total de 4 × 3 × 10 = 120 observaciones. Solo se midió el tiempo de la operación de multiplicación; la generación e inicialización de matrices se excluyó del cronometraje.

4.5 Herramientas de reproducibilidad

El proyecto utiliza Docker para aislar los entornos de ejecución de cada lenguaje y Snakemake como pipeline de automatización. Se incluye también un script run_native.sh para ejecución sin Docker.

5. Implementación paso a paso

Paso 1: Estructura del proyecto

Se creó la estructura de directorios del proyecto con carpetas separadas para cada implementación (implementations/cpp, implementations/go, implementations/java, implementations/python), el análisis estadístico (analysis/) y los resultados (results/).

Paso 2: Implementación del algoritmo

Se implementó el algoritmo de multiplicación naive en los cuatro lenguajes, asegurando que cada implementación use el mismo generador LCG con los mismos parámetros. Cada programa acepta argumentos de línea de comandos para el tamaño de matriz (--n), número de repeticiones (--reps), semilla (--seed) y archivo de salida (--output).

Paso 3: Contenerización con Docker

Se crearon Dockerfiles para cada lenguaje: gcc:13 para C++ (compilado sin optimización con -O0), golang:1.22 para Go, eclipse-temurin:21 para Java, y python:3.12-slim para Python.

Paso 4: Automatización con Snakemake

Se configuró un Snakefile que automatiza todo el pipeline: compilación/ejecución de benchmarks, integración de resultados en benchmark.csv, y ejecución del análisis estadístico.

Paso 5: Ejecución de benchmarks

Se ejecutaron los 120 benchmarks (4 lenguajes × 3 tamaños × 10 repeticiones) y se registraron los tiempos en archivos CSV individuales que luego se integraron en un único archivo benchmark.csv.

Paso 6: Análisis estadístico

Se ejecutó el script analyze.py que realiza el análisis descriptivo, ANOVA de dos factores con interacción, y comparaciones post-hoc Tukey HSD, generando tablas y gráficos.

Paso 7: Publicación en GitHub

Todo el proyecto, incluyendo código fuente, resultados y documentación, se publicó en el repositorio GitHub: github.com/jffp-alt/matrix-benchmark.

6. Resultados

6.1 Estadísticas descriptivas

La siguiente tabla muestra el tiempo promedio de ejecución (en segundos) y la desviación estándar para cada combinación de lenguaje y tamaño de matriz:

Lenguaje	N	Media (s)	Desv. Std (s)
C++	500	2.73	0.28
C++	1000	28.00	0.37
C++	1500	97.83	1.18
Go	500	0.32	0.02
Go	1000	4.72	0.08
Go	1500	21.42	0.63
Java	500	0.26	0.03
Java	1000	4.65	0.20
Java	1500	27.80	0.34
Python	500	12.46	0.26
Python	1000	101.88	1.82
Python	1500	342.08	4.55

Observaciones clave:

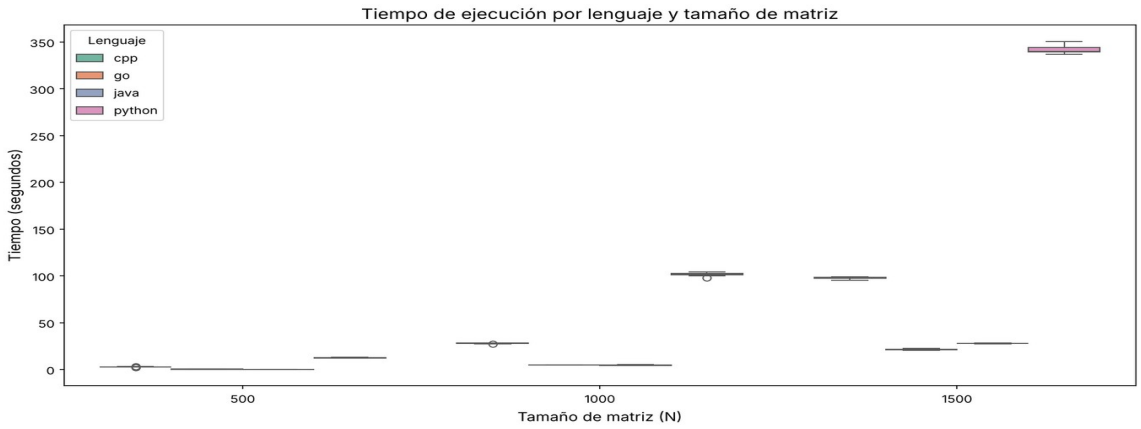
Java y Go son los lenguajes más rápidos, con tiempos similares para N=500 y N=1000.

Python es significativamente más lento que todos los demás: para N=1500, tarda 342 segundos (casi 6 minutos) frente a 21 segundos de Go.

C++ sin optimizaciones (-O0) muestra un rendimiento intermedio, más lento que Go y Java pero mucho más rápido que Python.

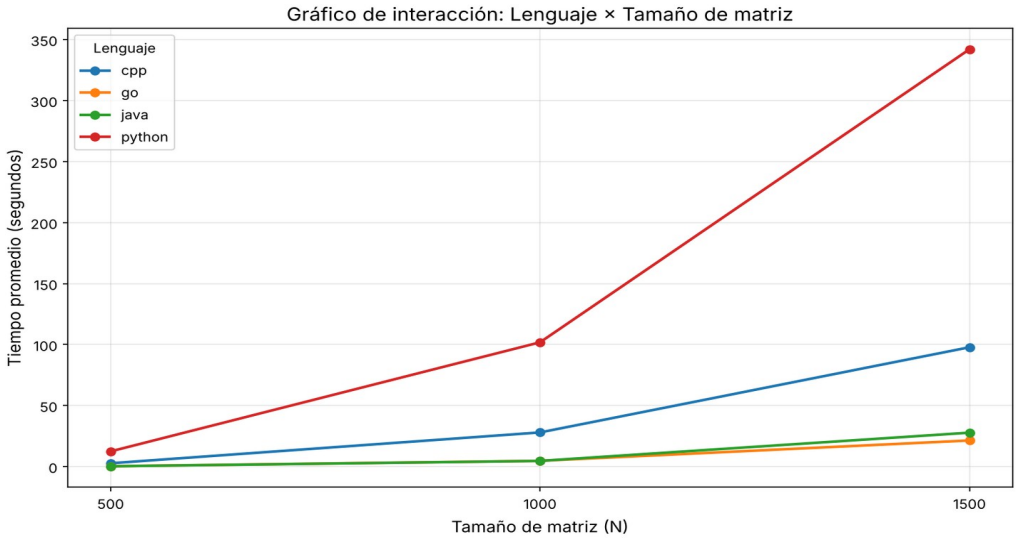
La diferencia entre lenguajes se amplifica con el tamaño de la matriz, indicando una fuerte interacción entre los factores.

6.2 Gráfico de distribución (Boxplot)



El boxplot muestra la distribución de tiempos por lenguaje y tamaño. Se aprecia claramente que Python domina los tiempos altos y que la variabilidad es baja en todos los lenguajes.

6.3 Gráfico de interacción



El gráfico de interacción evidencia que el efecto del lenguaje no es constante: Python crece exponencialmente con N, mientras que Go y Java mantienen tiempos bajos. Las líneas no son paralelas, confirmando la interacción significativa.

7. Análisis estadístico

7.1 ANOVA de dos factores

Se realizó un ANOVA de dos factores (lenguaje × tamaño de matriz) con interacción. Los resultados son:

Factor	F	p-valor	Significativo
Lenguaje	62,317.49	1.09e-174	Sí ***
Tamaño (N)	68,794.05	2.01e-168	Sí ***
Lenguaje × N	25,489.80	1.03e-167	Sí ***

Interpretación: Los tres efectos son altamente significativos ($p < 0.001$). Esto significa que tanto el lenguaje como el tamaño de la matriz afectan significativamente el tiempo de ejecución, y además existe una interacción significativa entre ambos factores (el efecto del lenguaje depende del tamaño de la matriz).

7.2 Comparaciones Post-hoc Tukey HSD

Las comparaciones Tukey HSD por lenguaje (colapsando tamaño) muestran:

Python vs todos los demás: Diferencia significativa ($p < 0.001$). Python es significativamente más lento.

C++ vs Go: No significativa ($p = 0.289$). En promedio, sus tiempos no difieren estadísticamente.

C++ vs Java: No significativa ($p = 0.344$).

Go vs Java: No significativa ($p = 0.999$). Son prácticamente equivalentes.

8. Conclusiones

Se rechaza la hipótesis nula (H_0). El análisis ANOVA confirma que existen diferencias estadísticamente significativas en el tiempo de ejecución entre los lenguajes ($F = 62{,}317$, $p < 0.001$).

Principales hallazgos:

1. Go y Java son los lenguajes más rápidos para este algoritmo, sin diferencias significativas entre ellos.

2. C++ sin optimizaciones (-O0) tiene un rendimiento intermedio, lo que sugiere que las optimizaciones del compilador son cruciales para su rendimiento.

3. Python (CPython) es 16x más lento que Go/Java para N=1500, confirmando la brecha de rendimiento del intérprete puro.

4. La interacción lenguaje × tamaño es significativa: la brecha de rendimiento se amplifica con matrices más grandes.

5. El experimento es completamente reproducible gracias a Docker, Snakemake, semillas fijas y el generador LCG idéntico en los cuatro lenguajes.

9. Criterios ACM Artifact Review & Badging

Functional: El código se ejecuta correctamente y produce los resultados documentados.

Reusable: Pipeline automatizado (Snakemake), entornos reproducibles (Docker), semilla fija, documentación completa.

Available: Publicado en GitHub con licencia MIT. Publicación en Zenodo pendiente para DOI.

10. Repositorio

GitHub: https://github.com/jffp-alt/matrix-benchmark

Licencia: MIT